

# AI-Enabled Smart Library Management and Resource Recommendation System

Project Proposal for UCS503P

Submitted to: Dr. Jeelani Asif

**Thapar Institute of Engineering and Technology**

Arnav Agarwal, CSED\*

Aradhya Goyal, CSED<sup>†</sup>

Amanjot Kaur Sidhu, CSED<sup>‡</sup>

29 August, 2026

## Abstract

University libraries manage large collections of books and other learning resources while handling student records, issue and return transactions, overdue items, fines, and resource availability. Conventional library management systems primarily provide storage, search, and CRUD-based operations, leaving several administrative activities manual and reactive. This project proposes a web-based Smart Library Management System that combines conventional library workflows with a lightweight multi-agent artificial intelligence layer. The proposed system consists of three specialized agents: a Library Intelligence Agent for natural-language resource discovery and recommendation, a Record Onboarding Agent for validating and importing student records, and a Library Monitoring Agent for overdue monitoring, notifications, and resource-demand analysis. The system integrates these agents with application APIs and a MySQL database so that AI-assisted operations are grounded in actual library data. The project emphasizes rapid incremental delivery, CI/CD, measurable evaluation, and practical deployment rather than novel AI research.

## 1 Higher-Order Goal

The higher-order goal of this project is to improve the efficiency and accessibility of university library services through software automation and data-driven resource management. The immediate engineering objective is to reduce the time and effort involved in common library workflows while providing students and administrators with a unified and deployable software platform.

The project uses artificial intelligence as an enabling technology rather than as a research objective. The primary focus remains on dependable software engineering, workflow automation, system integration, and measurable improvement in library operations.

## 2 Time-to-Value

The project is designed around rapid delivery and short feedback cycles.

- A usable minimum viable product (MVP) will be implemented during the initial iteration, including authentication, resource management, resource search, issue/return workflows, and database integration.

---

\*Roll No: 1024160010 Email: [aagarwal16e24@thapar.edu](mailto:aagarwal16e24@thapar.edu)

<sup>†</sup>Roll No: 1024160135 Email: [agoyal15e24@thapar.edu](mailto:agoyal15e24@thapar.edu)

<sup>‡</sup>Roll No: 1024160025 Email: [asidhu16e24@thapar.edu](mailto:asidhu16e24@thapar.edu)

- AI-assisted functionality will be introduced incrementally after the core library workflow is operational.
- CI will be established early so that every code change is automatically checked through linting, testing, and build validation.
- Subsequent iterations will add AI-assisted resource discovery, record onboarding, monitoring, and resource-demand analysis.
- Each iteration will produce a demonstrable increment that can be tested independently and improved using feedback.

The project therefore prioritizes fast feedback over attempting to implement the complete feature set at the beginning of the semester.

### 3 Problem Statement

University libraries handle a large number of resource records, student records, borrowing transactions, and overdue items. Although conventional library systems can store and retrieve these records, several operational problems remain.

- **Manual resource discovery:** Students may need to navigate through multiple search and filtering options to determine whether a suitable resource is available.
- **Manual record onboarding:** Importing large numbers of student or member records can require repetitive validation and manual duplicate checking.
- **Reactive overdue management:** Overdue resources are often identified only after staff manually inspect issue and due-date records.
- **Limited demand visibility:** Library administrators may not have an immediate way to identify frequently issued resources with insufficient available copies.
- **Data without assistance:** A conventional database stores library information but does not directly assist users in converting that information into actionable answers or recommendations.

**Why this matters now (contextual relevance):** A university library serves a large student population with continuously changing resource requirements. Improving resource discovery, record processing, overdue monitoring, and demand visibility can reduce repetitive administrative effort and improve the usability of existing library resources.

## 4 Proposed Solution

### 4.1 Overview

We propose a web-based Smart Library Management System consisting of a student interface, an administrator interface, application APIs, a MySQL database, and a controlled AI agent layer.

The system will provide the following core capabilities:

- **Resource management:** maintain records of books and other library-managed resources, including availability and issue status.
- **Student/member management:** create, update, search, and import student records.

- **Issue and return workflow:** record borrowing transactions, due dates, returns, and applicable fines.
- **Natural-language resource discovery:** allow students to search for suitable resources using ordinary language.
- **Automated monitoring:** detect overdue resources and initiate notification and fine-related workflows.
- **Demand analysis:** identify heavily issued resources with limited availability and generate purchase recommendations for administrators.

## 4.2 Core Workflow

The core workflow is divided into conventional software functionality and AI-assisted functionality.

1. A student authenticates and searches or requests a library resource.
2. The backend retrieves resource records and availability from the MySQL database.
3. For natural-language requests, the Library Intelligence Agent interprets the request and retrieves relevant information through controlled application tools.
4. The student issues an available resource, and the backend records the transaction and due date.
5. The Library Monitoring Agent periodically checks borrowing records and identifies resources approaching or exceeding their due dates.
6. The system generates reminders and updates relevant workflow records.
7. Historical issue data is analyzed to identify high-demand resources with comparatively low availability.
8. The administrator receives demand-based recommendations for acquiring additional copies or improving resource allocation.

## 4.3 Operational Constraints

- The system will use responsive web interfaces so that it remains usable on common student and administrative devices.
- AI functionality will rely on mature software libraries and available APIs rather than requiring novel model training.
- AI agents will operate through controlled application APIs or tools rather than unrestricted direct database access.
- Conventional database queries will be used whenever deterministic logic is sufficient; AI will be used where natural-language interaction, recommendation, summarization, or pattern interpretation provides additional value.

# 5 Solution Approach

This project is an engineering-focused software system. The implementation will emphasize modular design, reliable workflows, automated testing, deployment, and measurement rather than novel AI algorithm research.

## 5.1 Web Architecture

The proposed system follows a modular 3-tier web architecture integrated with a controlled AI agent layer:

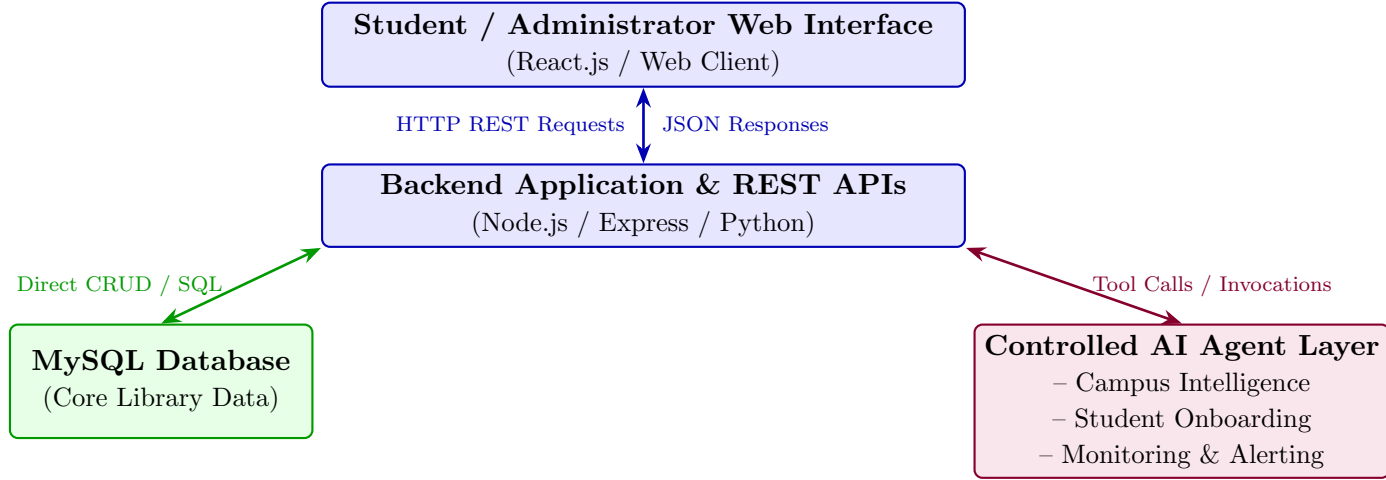


Figure 1: Modular 3-Tier Web Architecture with Integrated AI Agent Layer.

## 5.2 Library Intelligence Agent

The Library Intelligence Agent is intended for students and administrators. Its responsibilities include:

- interpreting natural-language resource requests,
- searching library resources through application APIs,
- providing availability-aware recommendations,
- answering common resource-related queries, and
- generating concise summaries from library data where appropriate.

## 5.3 Record Onboarding Agent

The Record Onboarding Agent assists administrators with bulk student or member imports. Its responsibilities include:

- uploading structured records,
- validating required fields,
- detecting likely duplicate records,
- inserting valid records through controlled backend operations, and
- generating an import summary for the administrator.

## 5.4 Library Monitoring Agent

The Library Monitoring Agent is responsible for scheduled monitoring and resource-management assistance. Its responsibilities include:

- detecting resources approaching their due date,
- detecting overdue resources,
- initiating reminder and fine-related workflows,
- analyzing issue frequency and resource availability, and
- identifying high-demand resources for purchase consideration.

## 5.5 AI Safety and Control

AI-assisted operations will be constrained by the backend application. Agents will not receive unrestricted SQL access, and database modifications will occur through predefined application APIs or tools.

## 5.6 CI/CD and Fast Delivery

CI/CD will support frequent and repeatable releases. The CI pipeline will install dependencies, execute linting and static checks, run automated tests, and deploy successful builds to staging/production environments.

# 6 Evaluation Criterion

The success of the project will be evaluated using measurable metrics related to the main workflows.

## 6.1 Primary Evaluation Metrics

- **Resource discovery time:** median time required for a user to locate a requested or suitable available resource. Target: less than or equal to 10 seconds for predefined test queries.
- **Duplicate detection precision:** proportion of records flagged as duplicates that are actually duplicates in a labeled test dataset. Target: at least 90%.
- **Overdue detection coverage:** proportion of overdue transactions correctly identified by the monitoring workflow. Target: at least 99% during controlled testing.
- **Demand recommendation accuracy:** proportion of high-demand/low-availability resources correctly identified according to predefined thresholds. Target: at least 90%.

## 6.2 Secondary Evaluation Metrics

- **API response time:** median response time for common non-AI application requests. Target: less than or equal to 500 ms under normal test load.
- **AI answer validity:** proportion of predefined library questions for which the agent returns information consistent with the database. Target: at least 90%.
- **Notification delay:** time between an overdue condition becoming eligible and generation of the corresponding notification.

- **System reliability:** availability of the deployed application during the pilot period. Target: at least 99%.
- **Test coverage:** percentage of critical backend logic covered by automated tests. Target: at least 70%.

### 6.3 Baseline and Validation Plan

Evaluation will use controlled datasets and predefined test scenarios. For resource discovery, the team will measure the time required to locate resources using the conventional interface and compare it with the natural-language interaction. For onboarding, a labeled dataset containing valid, invalid, and duplicate records will be created to measure processing time and duplicate-detection precision. For monitoring, synthetic borrowing records with known due dates and overdue states will be used to measure detection coverage and notification latency. For demand analysis, historical or synthetic issue records will be used to define high-demand resources using predetermined thresholds. The resulting recommendations will then be compared against the expected labels.

## 7 Scalability

The system is designed to scale in both theoretical and practical terms.

### 7.1 Logical Scalability

- Frontend and backend responsibilities are separated.
- Backend APIs are designed to be stateless where practical.
- Database queries will use indexing and pagination for frequently accessed records.
- AI agents are modular and can be extended or replaced without rewriting the complete application.

### 7.2 Operational Deployability

- The application can be containerized using Docker.
- A managed relational database can be used for deployment.
- CI/CD supports repeatable staging and production deployments.
- The architecture can support a larger number of users and records without requiring a fundamental redesign.

## 8 Engine Availability Heuristic

The project relies on mature and readily available software engines rather than requiring fundamental algorithmic research.

- A modern web framework for frontend and backend development.
- REST API infrastructure for application integration.
- MySQL or another managed relational database.
- Authentication and role-based authorization libraries.

- Mature LLM or AI APIs and supporting agent frameworks.
- Automated testing frameworks for unit and integration testing.
- GitHub Actions or an equivalent CI runner.
- Docker and a platform-hosted or cloud deployment target.

## 9 Project Scope and Deliverables

### 9.1 Initial MVP

- Student and administrator authentication.
- Role-based access control.
- Library resource and student record management.
- Resource search and availability display.
- Issue and return workflow.
- Basic due-date and fine tracking.
- MySQL database integration.
- Backend API and frontend integration.
- CI pipeline with automated tests and linting.

### 9.2 Subsequent Deliverables

- Natural-language resource discovery.
- AI-based resource recommendation.
- Bulk student/member onboarding with duplicate detection.
- Automated overdue monitoring and notifications.
- Demand analysis for frequently issued and low-availability resources.
- Purchase recommendations for library administrators.
- Administrative analytics and reporting.
- Search, filtering, and database performance improvements.

### 9.3 Final Prototype

The final prototype will integrate the major modules into a deployable system and demonstrate end-to-end student and administrator workflows, all three AI-assisted modules, automated testing and CI/CD, measurable evaluation results, scalability-oriented implementation decisions, and deployment on an accessible hosting environment.

## 10 Timeline

| Week | Primary Task                              | Expected Deliverable                            |
|------|-------------------------------------------|-------------------------------------------------|
| W1   | Project setup and Requirements            | Requirements, repository, initial architecture  |
| W2   | Database and backend foundation           | Database schema, APIs, CI pipeline              |
| W3   | Authentication and core library workflows | Login, roles, resource and student modules      |
| W4   | Proposal completion and MVP integration   | Initial integrated MVP                          |
| W5   | Search, issue, return, and fine workflows | Functional library transaction system           |
| W6   | Testing and deployment                    | Automated tests and staging deployment          |
| W7   | First prototype                           | Working MVP demonstration                       |
| W8   | Feedback and improvement plan             | Measured feedback and prioritized improvements  |
| W9   | Library Intelligence Agent                | Natural-language resource discovery             |
| W10  | Recommendation functionality              | Availability-aware resource recommendations     |
| W11  | Record Onboarding Agent                   | Bulk import and duplicate detection             |
| W12  | Library Monitoring Agent                  | Overdue detection and notification workflow     |
| W13  | Demand analysis                           | High-demand/low-availability detection          |
| W14  | Integration and performance testing       | Integrated second-stage system                  |
| W15  | Second prototype                          | Second prototype demonstration                  |
| W16  | Refinement and final testing              | Bug fixes, optimization, final evaluation       |
| W17  | Final prototype and presentation          | Final deployed system, report, and presentation |

## 11 Risks and Mitigations

- **AI hallucination or incorrect resource information:** AI responses will be grounded in current database records through controlled retrieval tools and predefined APIs.
- **Excessive project scope:** The core library workflow will be completed before AI features are added, and later functionality will be delivered incrementally.
- **Duplicate detection errors:** Potential duplicates will be flagged for human review instead of being automatically deleted.
- **AI service unavailability:** Conventional keyword-based resource search will remain available as a fallback.
- **Unauthorized automated actions:** Agents will operate through role-controlled APIs, and irreversible operations will require explicit backend authorization.
- **Insufficient evaluation data:** Synthetic and labeled library datasets will be prepared early so that evaluation does not depend entirely on access to live institutional data.
- **Deployment instability:** A staging environment and CI/CD pipeline will be established early to make deployments repeatable.



- **Database performance degradation:** Frequently accessed fields will be indexed and database operations will use pagination and optimized queries.

## 12 Summary

This project proposes a deployable AI-enabled Smart Library Management System that combines conventional library workflows with carefully scoped AI-assisted functionality. The system addresses practical problems in resource discovery, record onboarding, overdue monitoring, and resource demand analysis.

The project aligns with software engineering requirements by prioritizing rapid time-to-value, incremental delivery, CI/CD, mature engineering engines, scalability, and measurable evaluation. A working MVP will be produced early and subsequently extended through AI-assisted resource discovery, onboarding, monitoring, and demand-based recommendations.

The system will be evaluated using measurable workflow, accuracy, performance, and reliability metrics rather than relying only on qualitative claims. The project therefore focuses on building and validating a practical, scalable, and deployable library-management platform rather than pursuing novel AI research.